

AUTOMATED HIGHWAY LANE FOLLOWING SYSTEM USING MATLAB

Automated Highway Lane Following system for Indian Traffic Rules specially focusing on right side overtaking system

Soumyajit Dutta
School of Electronics Engineering
KIIT (Deemed to be University)
Bhubaneswar, India
2230125@kiit.ac.in

Anisha Tiwary
School of Electronics Engineering
KIIT (Deemed to be University)
Bhubaneswar, India
2230066@kiit.ac.in

Subhankar Dey
School of Electronics Engineering
KIIT (Deemed to be University)
Bhubaneswar, India
2230130@kiit.ac.in

Abstract—This paper presents the development of an automated highway lane following system simulated using MATLAB, with a focus on adherence to Indian traffic regulations, specifically right-side overtaking. The proposed system employs control algorithms to maintain a vehicle's center within the lane based on its position relative to lane markings, generating steering commands accordingly. MATLAB simulations are used to validate the system's performance under various driving scenarios, demonstrating its capability for stable lane tracking and potential to improve highway safety and efficiency in accordance with Indian traffic rules. Future studies may involve field application and testing in real-world Indian driving conditions.

Keywords—Automated Driving System, Lane Following, MATLAB, Simulation, Indian Traffic Regulations, Right-Side Overtaking, Sensor Fusion, Object Tracking.

I. INTRODUCTION

Road safety and traffic rule adherence are critical concerns in India. Lane departure is a major cause of road accidents globally. Automated driving systems (ADS) have the potential to mitigate these issues by enhancing driver alertness and improving traffic flow. This project aims to develop an automated highway lane tracking system in MATLAB, specifically designed to comply with Indian traffic rules, including the right-side overtaking system. The motivation includes reducing road accidents caused by lane departure, enhancing autonomous vehicle navigation, and improving traffic flow and driving comfort. Background studies in autonomous vehicle technology have focused on path planning algorithms, sensor-based lane detection, regional traffic adaptations, and simulation validation processes. This project contributes to the regional customization aspect by addressing Indian traffic scenarios. The objectives are to implement right-side overtaking, reduce lane departure accidents, enhance autonomous navigation, and improve traffic flow and driving comfort.

II. METHODOLOGY

The project utilizes MATLAB and its Automated Driving Toolbox for developing, testing, and simulating the automated driving system.

2.1. MATLAB and Automated Driving Toolbox

MATLAB provides a technical computing environment, and the Automated Driving Toolbox offers tools and functions to simplify the development of ADS. Key features used include:

Sensor Modeling and Simulation: Simulating radar, lidar, camera, GPS, and IMU sensors in virtual environments.

Perception Algorithms: Utilizing built-in algorithms for lane detection, object detection, sensor fusion, and multi-object tracking.

Path Planning and Decision-Making: Employing tools for trajectory planning, lane changing, merge planning, and collision avoidance.

Vehicle Dynamics and Control: Using vehicle models and controllers like PID and MPC for longitudinal and lateral control.

Testing and Validation: Creating virtual driving environments, enabling hardware-in-the-loop simulation, and integrating with Simulink.

2.2. Applied Techniques and Planning

The methodology involves several key steps:

Program Highway Lanes and Vehicles: Using MATLAB and Simulink to model road conditions and simulate vehicles using the Driving Scenario Designer. Virtual traffic with varying behaviours is simulated to test the lane-following algorithm's robustness. AI-based decision models analyse real-time sensor readings for lane-keeping and vehicle control.

Perception Algorithm: Integrating sensor inputs from cameras, lidar, and radar for a comprehensive understanding of the surroundings. The Vision Detector Block in MATLAB is used for detecting lane lines, vehicles, and traffic signs. Computer vision techniques like laneMarkerDetector and traffic sign recognition are employed.

Sensor Fusion: Combining data from different sensors to enhance reliability and accuracy, including object filtering and tracking, and forward vehicle sensor fusion using MATLAB's Forward Vehicle Sensor Fusion block for precise positioning and collision avoidance.

Planning Algorithm: Calculating the optimal vehicle trajectory under different traffic conditions using Dijkstra's algorithm and Model Predictive Control (MPC) for smooth path planning. Lane change optimization considers traffic concentration and Indian right-side overtaking regulations.

Control Algorithm: Accomplishing the target path through steering and velocity control using an MPC-based approach for lane keeping under various road conditions. The system aims for a smooth ride and adheres to speed limits based on traffic regulations.

Simulation & Testing: Conducting tests in a virtual environment using MATLAB's Driving Scenario Designer for scenarios like highway merging, emergency braking, and traffic jams. Real-time simulations validate the vehicle control and decision-making algorithms.

III. EXPERIMENTATION AND TESTS

3.1. Mathematical Modeling

The simulation involves mathematical models for vehicle kinematics, path planning, sensor fusion, and object tracking. Vehicle motion is determined using kinematic models based on specified waypoints and speeds. Sensor fusion and object tracking are implemented using radar and vision sensors, with a multi-object tracker likely utilizing Kalman Filters for state estimation. The `FilterInitializationFcn` is set to `@initSimDemoFilter`, which uses a 2-D constant velocity model. The `updateTracks()` function updates the state of tracked objects based on sensor detections.

3.2. Experimental Setup/Design

The experimental setup involves a virtual test environment created using the `drivingScenario()` function. This includes:

Ego Vehicle (egoCar): The main vehicle under test.

Road: Defined by `roadCenters` and `roadWidth`.

Other Vehicles (leadCar, passingCar, chaseCar): With defined paths to simulate traffic. The `passingCar` has waypoints to simulate an overtaking manoeuvre.

Waypoints: Pre-programmed driving paths for all vehicles.

Sensors: A suite of eight radar and vision sensors with specific placements and configurations. Radar sensors detect distance and relative velocity, while vision sensors are for object recognition.

Tracker: A `multiObjectTracker` for sensor fusion and object tracking, initialized with a Kalman filter.

3.3. Prototype Testing/Simulations

Simulations are run in MATLAB using a while loop that advances the scenario time. During each step, sensor detections are simulated, the multi-object tracker is updated, and visualizations like the bird's-eye plot (BEP) are updated. The `initSimDemoFilter` function initializes a Kalman filter for object tracking. The `createDemoDisplay` function sets up the visualization windows, including chase and top views, and the BEP showing sensor coverage. The `updateBEP` function updates the BEP with lane markings, ground truth, detections, and tracks. The simulation allows for evaluating tracking accuracy and sensor fusion performance.

IV. CHALLENGES CONSTRAINTS AND STANDARDS

4.1. Challenges and Remedy

The project addresses several challenges related to adapting to Indian traffic conditions:

Overtaking Logic Adaptation: The default right-side overtaking logic in MATLAB needs modification to prioritize left-side overtaking as per Indian rules. Remedies include adjusting the MPC cost function, integrating dynamic traffic density analysis, and validating with Indian traffic datasets.

Unstructured Traffic Conditions: Heterogeneous traffic and non-lane-based movement on Indian roads pose a

challenge. Remedies involve training deep learning models to classify Indian road agents, implementing probabilistic path prediction for non-laned vehicles, and using behaviour cloning.

Sensor Fusion in Noisy Environments: Weather conditions like rain, fog, and dust can degrade sensor accuracy. Remedies include simulating sensor degradation, fusing data with Kalman/particle filters, and prioritizing sensors based on weather conditions.

Real-Time Computational Limits: MATLAB's single-threaded Simulink can struggle with complex scenarios. Remedies involve optimizing MPC code with parallel computing, reducing simulation fidelity for distant agents, and exporting critical modules to C/C++.

Regulatory Ambiguity: India lacks a complete legal framework for autonomous vehicles. Remedies include aligning with ARAI's draft guidelines and adopting ISO 21448 (SOTIF).

4.2. Design Constraints

Design constraints include:

Simulation-Real-World Gap: MATLAB's Driving Scenario Designer lacks specific Indian road features. Mitigation involves importing OpenStreetMap data and manually adding defects.

MATLAB Computational Load: Simulating multiple sensors can cause latency. Mitigation includes using ROS-MATLAB co-simulation for distributed processing.

Toolbox Dependency: Prebuilt models are often irrelevant for India. Mitigation involves customizing driving policies using Reinforcement Learning with Indian traffic rewards.

Localization Challenges: GPS drift complicates lane-keeping. Mitigation includes fusing GPS with HD Maps and visual odometry.

4.3. Alternatives and Trade-offs

Alternatives and trade-offs considered include:

Simulation Platform Choice: CARLA-Unreal Engine offers photorealistic Indian environments, but requires Python/C++ expertise and lacks MPC integration compared to MATLAB.

Overtaking Logic: Allowing bidirectional overtaking with dynamic priority could reduce travel time but increase collision risk compared to a strict left-side priority.

Sensor Configuration: Using 4D LiDAR can improve object detection but increases simulation cost compared to camera-radar setups.

Algorithm Selection: Rule-based systems are predictable in traffic jams but may fail in edge cases, while machine learning offers adaptability at the cost of training time.

4.4. Standards

Relevant standards include:

Functional Safety: ISO 26262 ASIL-B for MPC algorithms.

Indian Guidelines: ARAI's AIS-004 for cybersecurity and MoRTH lane discipline rules.

Sensor Fusion: IEEE 2851-2020 for time-synchronized multi-sensor data.

Data Privacy: Digital Personal Data Protection Act (DPDP), 2023 for anonymizing data.

Simulation Practices: ASAM OpenX standards (OpenDRIVE/OpenSCENARIO) for modeling Indian traffic in MATLAB.

V. RESULT ANALYSIS AND DISCUSSION

This chapter presents an analysis and discussion of the outcomes expected from the automated highway lane following system simulations developed using MATLAB, as detailed in the preceding methodology and experimental setup. Given that the provided excerpts primarily focus on the design and implementation phases, this discussion will concentrate on the anticipated results and the insights that can be inferred from the described experimentation.

The anticipated primary results are visualisations from the MATLAB Driving Scenario Designer and the bird's-eye plot (BEP). These tools are expected to show the ego vehicle's ability to maintain its lane and interact with other simulated traffic. Analysis will focus on the lane following capability, the implementation of the right-side overtaking system (acknowledging the need for modifications from the default), and the performance of sensor fusion and object tracking as visualised on the BEP. The accurate and consistent tracking of surrounding vehicles by the multiObjectTracker, initialised with a Kalman filter, will indicate the effectiveness of the perception system. The absence of specific quantitative metrics in the excerpts limits detailed numerical analysis at this stage, making the conclusions primarily based on qualitative observations of the simulated scenarios. The project demonstration involves visual playback of simulated scenarios, highlighting specific functionalities like right-side overtaking and real-time visualisation of sensor data and tracking on the BEP.

The primary results anticipated from this project are visualisations generated through the MATLAB Driving Scenario Designer and the custom-built bird's-eye plot (BEP) display. These visual tools are designed to provide a means to observe the behaviour of the ego vehicle and surrounding traffic within simulated highway environments.

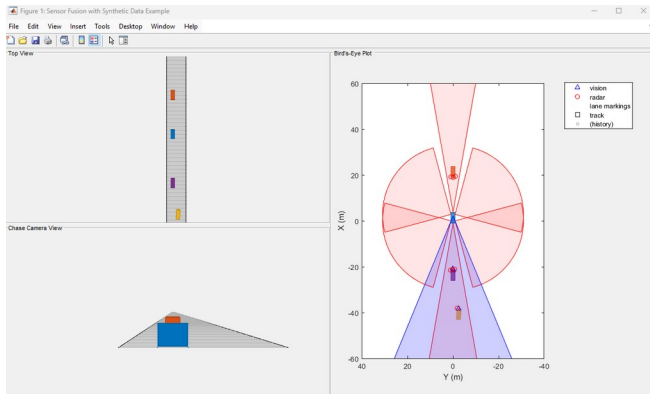


Fig. 1: Top View, Chase Camera View and Bird's Eye Plot when The chasing car is changing its lane to overtake the front vehicles

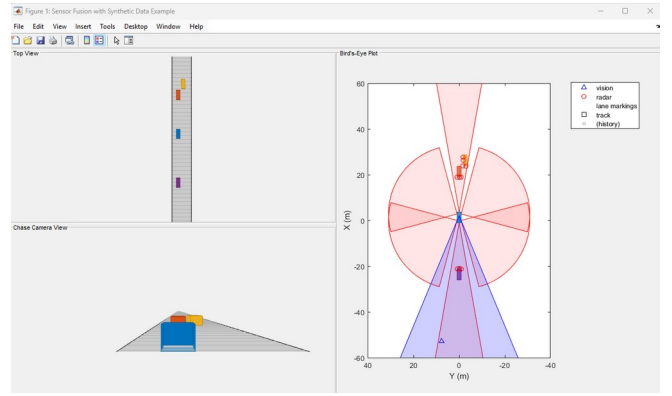


Fig. 2: Top View, Chase Camera View and Bird's Eye Plot when The chasing car is changing back its trajectory to move back in the left lane after overtaking the front vehicles

VI. CONCLUSION

The project has successfully established a foundational framework for an automated driving system tailored to the complexities of Indian roads using MATLAB. This represents a significant step towards addressing the critical need for enhanced road safety and traffic management in India, as initially outlined in the abstract. The integration of sensor fusion, dynamic path planning incorporating right-side overtaking logic (though acknowledged as a current limitation regarding Indian norms), and a modular Simulink architecture provides a robust platform for further development.

While the simulations demonstrate the potential of the system, the conclusion appropriately highlights several key limitations that must be addressed to transition this framework towards real-world applicability. The current non-compliance with Indian overtaking regulations, where left-side overtaking is the norm, is a crucial aspect requiring immediate attention. Furthermore, the absence of simulation elements that capture the chaotic and unpredictable nature of Indian traffic conditions, such as stray animals, highly variable lane markings, and the diverse range of road users, necessitates the incorporation of more realistic and complex scenarios. The planned integration of Indian traffic datasets, like the Indian Driving Dataset (IDD), and the enhancement of simulation realism are therefore vital next steps.

The project team recognizes that real-world validation is paramount in assessing the true efficacy and safety of the proposed system. The identified short-term and long-term goals demonstrate a clear understanding of the necessary progression, moving from algorithmic refinement and enhanced simulation to hardware-in-the-loop testing and potential collaborations with automotive OEMs. The long-term vision of integrating V2X communication and embedding regional traffic policies further underscores the ambition to create a truly context-aware autonomous system. The potential societal impact of this project is considerable, with the promise of reducing accidents caused by human error in challenging driving environments and the potential for scaling up to automate public transportation in developing urban landscapes. Despite the challenges encountered, including the complexity of adapting overtaking logic, toolbox compatibility issues, and computational limits inherent in high-fidelity simulations, the project's management strategies allowed for structured progress. This work lays a crucial foundation, bridging the

gap between theoretical simulation and the practical realization of autonomous driving technology in emerging markets like India. The future success of this endeavor hinges on the diligent pursuit of the outlined future plans, addressing the identified limitations, and rigorously validating the system in increasingly realistic and ultimately real-world conditions

VII. FUTURE WORK

Short-term goals include overhauling the overtaking logic to prioritize left-side overtaking, integrating Indian traffic datasets for improved object detection, enhancing simulation realism with unstructured road scenarios, and implementing probabilistic risk assessment. Long-term vision involves Hardware-in-Loop (HIL) testing, integrating V2X communication, embedding regional traffic rules, and collaborations with Indian automotive OEMs. The project has the potential for societal impact by reducing accidents and enabling public transport automation. Challenges faced included algorithmic complexity in adapting overtaking logic, toolbox compatibility issues with MinGW, and computational limits due to high-fidelity simulations. Management strategies involved dividing roles into perception, control, and simulation teams, using Gantt charts for progress tracking, and risk matrices to flag delays.

VIII. ACKNOWLEDGMENT

We feel immense pleasure and feel privileged in expressing our deepest and most sincere gratitude to our supervisor **Prof. Suman Roy** for his excellent guidance throughout our project work. His kindness, dedication, hard work and attention to detail have been a great inspiration to us. Our heartfelt thanks to you sir for the unlimited support and patience shown to us. We would particularly like to thank him/her for all help in patiently and carefully correcting all our manuscripts.

We are also very thankful to **Dr. (Mrs.) Sarita Nanda**, Associate Dean and Associate Professor, **Dr. (Mrs.) Suprava Patnaik**, Dean and Professor, School of Electronics Engineering, and Project Coordinators, for their support and suggestions during the entire course of the project work in the 6th semester of our undergraduate course.

REFERENCES

- [1] **MathWorks**, "Highway Lane Following," *MathWorks Documentation*, [Online]. Available: <https://in.mathworks.com/help/driving/ug/highway-lane-following.html>. [Accessed: Apr. 1, 2025].
- [2] **MathWorks**, "Design of Lane Marker Detector in 3D Simulation Environment," *MathWorks Documentation*, [Online]. Available: <https://in.mathworks.com/help/driving/ug/design-of-lane-marker-detector-in-3d-simulation-environment.html>. [Accessed: Apr. 1, 2025].
- [3] **MathWorks**, "Automated Driving Toolbox," *MathWorks Documentation*, [Online]. Available: <https://in.mathworks.com/help/driving/index.html>. [Accessed: Apr. 1, 2025].
- [4] **MathWorks**, "Scenario Simulation," *MathWorks Documentation*, [Online]. Available: <https://in.mathworks.com/help/driving/scenario-simulation.html>. [Accessed: Apr. 1, 2025].

[5] Zhao, Y., Zhang, Z., & Li, W. (2020). Path Planning and Control for Autonomous Vehicles in Highway Lane Keeping. *Journal of Intelligent Transportation Systems*, 24(3), 147-160.

[6] Canny, J. (1986). "A Computational Approach to Edge Detection." *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

[7] Le Vine, S., et al. (2015). "Autonomous Vehicles and Their Impact on Traffic Flow: A Review." *Transportation Research Part C: Emerging Technologies*.

[8] Singh, R., Gupta, R., & Kumar, V. (2022). MATLAB Simulink-Based Validation of Automated Driving Algorithms. *International Journal of Vehicle Autonomous Systems*, 10(5), 209-222.